Data Structures and Algorithms -3 (25CS2103E)

Subset Optimization Tool using SOS Dynamic Programming

Team Members

Vishnu Vardhan – 2520030084

Sandeep - 2520030523

Under the Esteemed Guidance of

Dr CH Anuradha department of computer science and engineering

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

Many problems ask for the sum of values over all subsets of a given set, repeated for many queries. Enumerating subsets by brute force costs O(2^|x|) for every query, which becomes too slow as queries grow.

This project develops a Subset Optimization Tool based on Sum Over Subsets (SOS) Dynamic Programming. The input array is preprocessed once in O(n·2^n) time, after which every subset-sum query is answered in O(1).

The tool checks every answer against brute force, draws the subset lattice, and plots execution time. In our test run (n = 14, 10,000 queries) SOS DP was about 79 times faster than brute force.

Introduction

- **Subsets as bitmasks:** a set of n elements has 2^n subsets; each subset is stored as an n-bit mask.
- **Goal:** for a mask x, find F[x] = sum of A[s] over every subset s of x.
- **SOS DP idea:** process one bit at a time; if bit i is set in mask, add F[mask without bit i].
- **Cost:** O(n·2^n) preprocessing, then O(1) per query (brute force: O(2^|x|) per query).
- **Uses:** bitmask DP, counting subsets, inclusion–exclusion, subset convolution.

Literature Survey

- **Rota (1964):** Möbius functions on partially ordered sets; the zeta transform on the subset lattice is the mathematical basis of SOS DP.
- **Yates (1937):** Fast algorithm for factorial experiments; it uses the same one-dimension-at-a-time sweep as SOS DP.
- **Björklund, Husfeldt, Kaski, Koivisto (2007):** "Fourier Meets Möbius: Fast Subset Convolution", STOC; uses the fast zeta and Möbius transforms in O(n·2^n).
- **Cormen et al.:** Introduction to Algorithms; dynamic programming fundamentals (optimal substructure, overlapping subproblems).
- **Competitive-programming tutorials:** Bitmask formulation of SOS DP (for example, Codeforces SOS DP tutorial), used for the implementation here.

Gap addressed: existing material explains the algorithm only. This project adds verification, subset-lattice visualization and a performance comparison.

Problem Statement and Objectives

Problem statement: Develop a subset optimization tool using Sum Over Subsets (SOS) Dynamic Programming that efficiently answers multiple subset-based queries and computes aggregate subset values, demonstrates its advantage over brute force, and visualizes subset relationships and execution performance.

Input
Integer set: array A of 2^n values (one per subset)
and a list of queries (masks)

→

Output
Subset sums: F[x] = sum of A[s]
for all subsets s of query mask x

Objectives

- Implement SOS DP to compute the subset sum F[mask] for every mask.
- Answer many subset-sum queries efficiently (O(1) each after preprocessing).
- Compare with brute force and show where SOS DP is faster.
- Visualize the subset lattice and the execution time of both methods.

Algorithm and Complexity

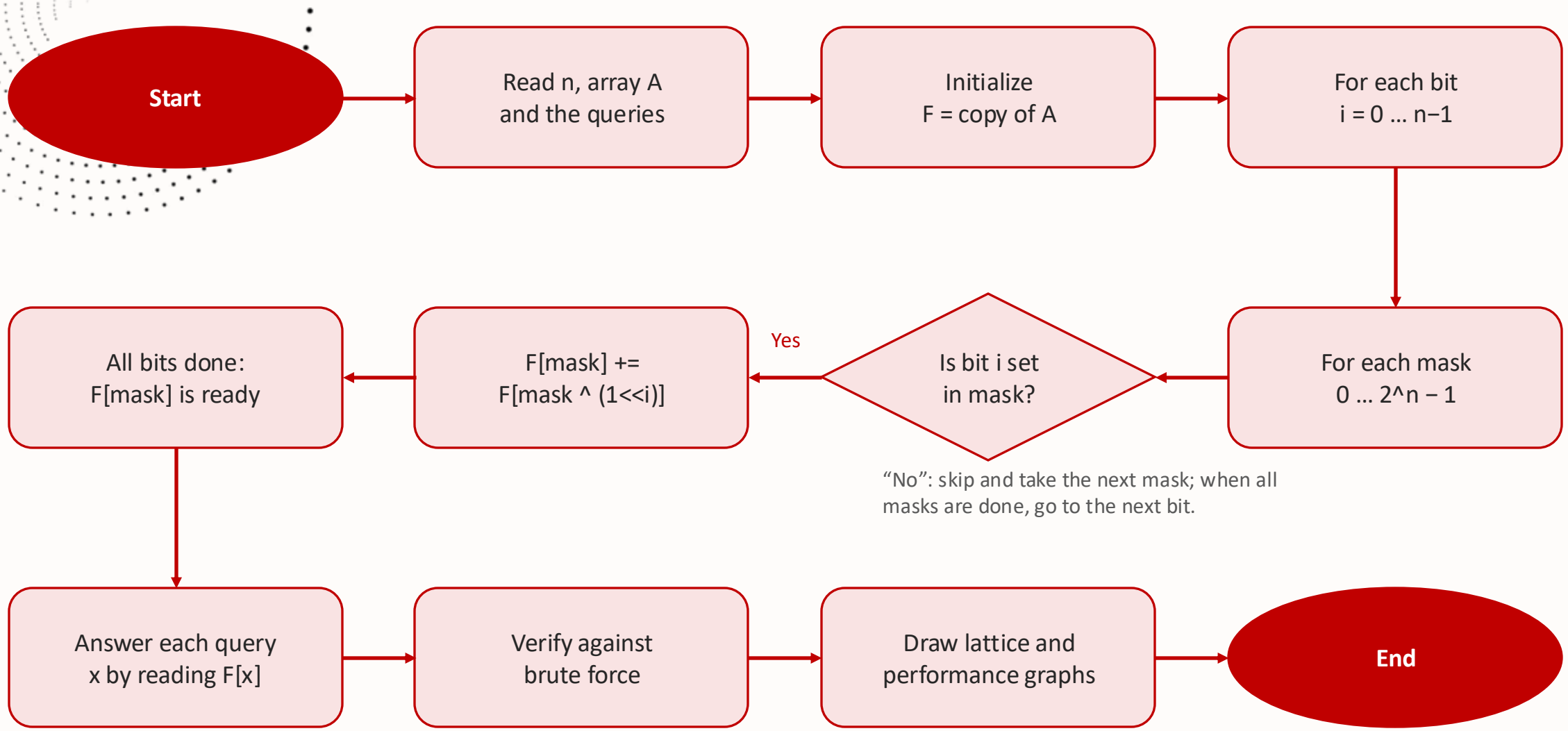
```
def sos_dp(A, n):
    F = A[:]
    for i in range(n):
        for mask in range(1 << n):
            if mask & (1 << i):
                F[mask] += F[mask ^ (1 << i)]
    return F

def brute_force(A, x):
    total, sub = A[0], x
    while sub:
        total += A[sub]
        sub = (sub - 1) & x
    return total
```

Method	Preprocess	Per query
Brute force	none	O(2^|x|)
SOS DP	O(n·2^n)	O(1)

- **Total for Q queries:** brute force O(Q·2^n) in the worst case; SOS DP O(n·2^n + Q).
- **Space:** O(2^n) for the array F.
- **Why it works:** after step i, F[mask] holds the sum over subsets that differ from mask only in bits 0..i.

Flow Chart


Start
Read n, array A and the queries
Initialize F = copy of A
For each bit i = 0 … n−1
For each mask 0 … 2^n − 1
Is bit i set in mask?
Yes
F[mask] += F[mask ^ (1<<i)]
All bits done: F[mask] is ready
Answer each query x by reading F[x]
Verify against brute force
Draw lattice and performance graphs
End
"No": skip and take the next mask; when all masks are done, go to the next bit.

Results: Sample Run

Input array and SOS result (n = 3)

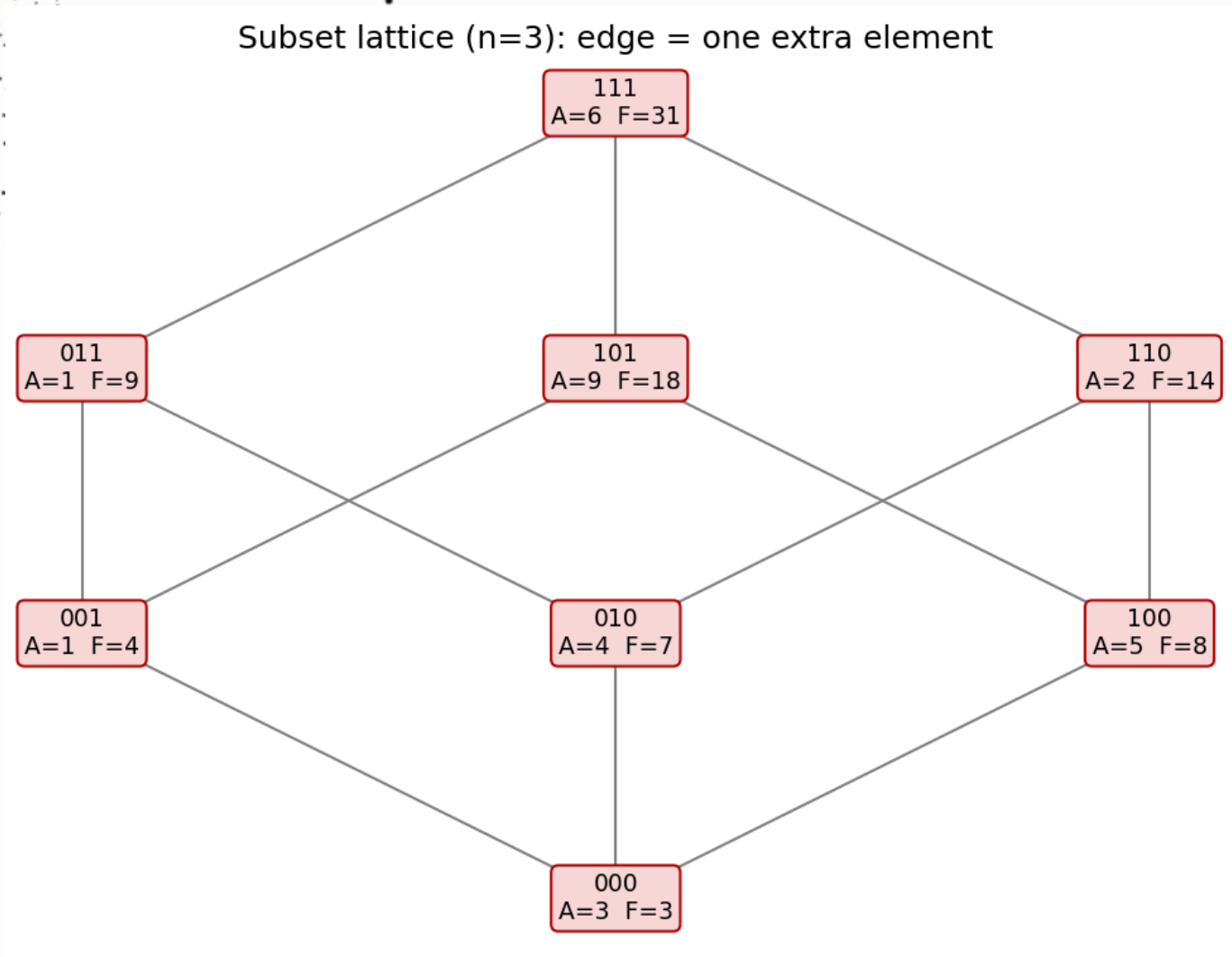
mask	A[mask]	F[mask]
000	3	3
001	1	4
010	4	7
011	1	9
100	5	8
101	9	18
110	2	14
111	6	31

Query results

Query mask	SOS DP	Brute force
111	31	31
101	18	18
011	9	9
100	8	8

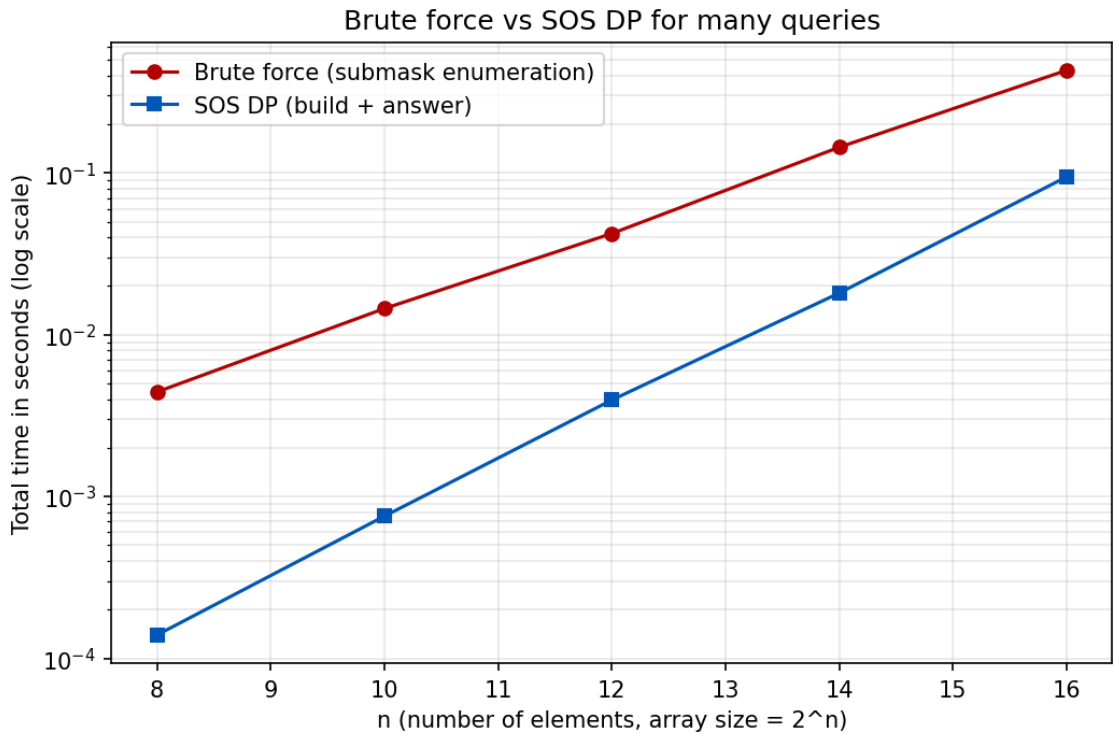
- **Check:** F[101] = A[000] + A[001] + A[100] + A[101] = 3 + 1 + 5 + 9 = 18.
- All SOS answers match brute force; the code asserts this on every run.

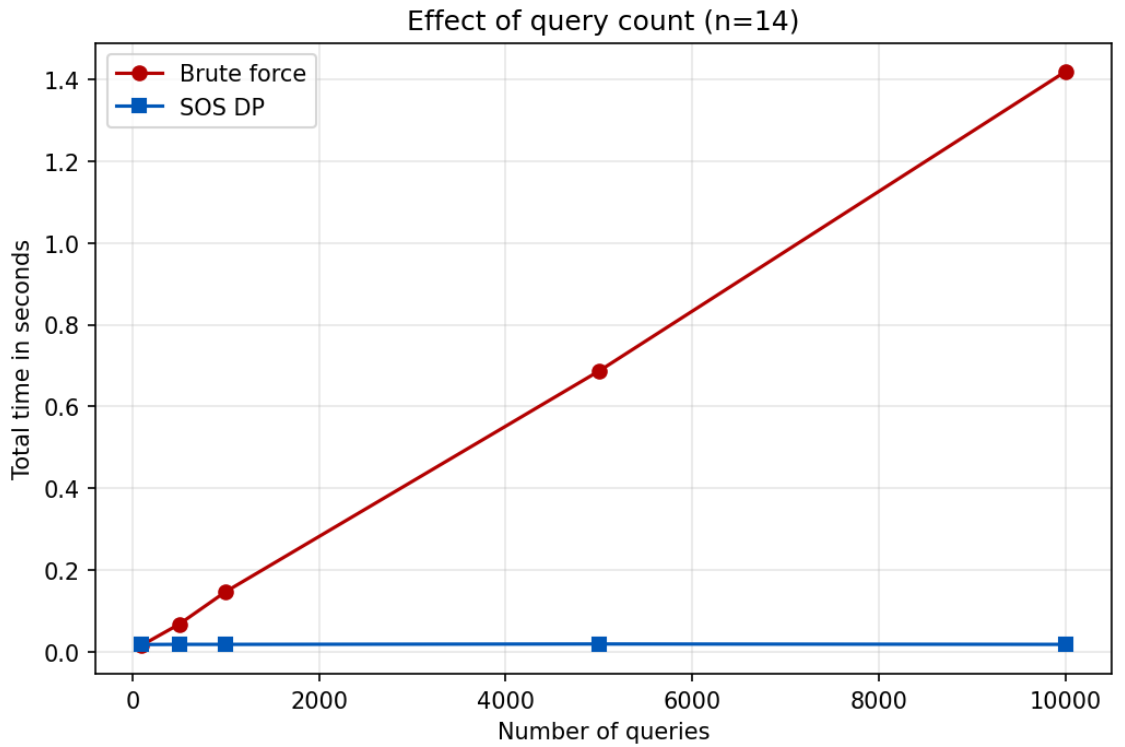
Results: Subset Lattice



- **Node:** a subset, written as a bit string (A = given value, F = SOS subset sum).
- **Edge:** joins two subsets that differ by exactly one element.
- **Reading F:** F[mask] adds A of the node itself and every node below it.
- **Top node:** F[111] = 31, the sum of all eight A values.

Results: Performance





- **Fixed 1000 queries:** SOS DP is faster at every n tested (about 32× at n = 8, about 5× at n = 16) because brute force also pays only for submasks.
- **n = 14, growing queries:** SOS time stays near 0.018 s; brute force grows with queries (0.9× at 100 queries, about 79× at 10,000).

Hardware and Software requirements

Hardware

- Standard PC or laptop
- 4 GB RAM or more
- Any modern dual-core CPU
- No GPU needed

Software

- Python 3.8 or later
- matplotlib (pip install matplotlib)
- Any IDE or terminal (VS Code, PyCharm, IDLE)
- Windows, Linux or macOS

Conclusion

- SOS DP computes every subset sum in O(n·2^n), then answers each query in O(1).
- Its answers matched brute force in every test run.
- The advantage grows with the number of queries: about 79× faster at 10,000 queries (n = 14), while for very few queries brute force can be as fast.
- The lattice diagram and time graphs make the subset structure and the performance gap easy to see.
- Limitation: memory grows as 2^n, so n is limited to about 20–25 in practice.

Future Scope

- Superset sums and the inverse (Möbius) transform to recover A from F.
- Subset convolution built on the same transforms.
- Other aggregates: count, maximum, minimum, XOR.
- A GUI or web interface that animates the DP step by step.
- Faster versions using NumPy or C++ for larger n.

(DEEMED TO BE UNIVERSITY)

Thank you